Q2 | Connected k-core index

Revised 5 October 2026

15 marks. Design an index for connected k-core search and core-connectivity queries.

Problem statement

For a simple undirected graph G, the k-core is the unique maximal induced subgraph in which every vertex has at least k neighbours within the subgraph. A connected k-core is one connected component of that k-core. A vertex's core number is the largest k for which it belongs to the k-core.

Implement **KCoreIndex.build()**, **query(k, v)** and **max_k(u, v)**:

build(): preprocess the graph once, store the index and return self.

query(k, v): return all vertex IDs in v's connected k-core, **in ascending order**, with no duplicates. Return [] if v is absent from the k-core.

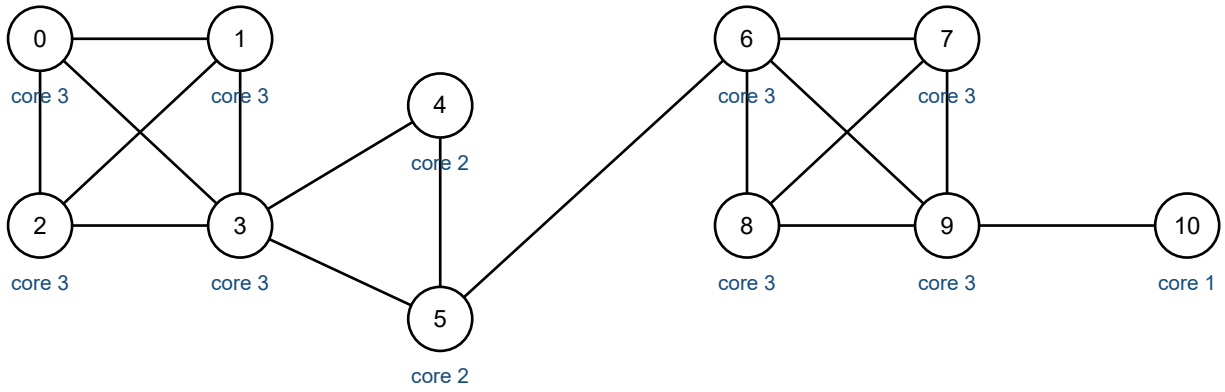
max_k(u, v): return the largest k for which u and v belong to the same connected k-core, or -1 if they are disconnected in G. If u = v, return that vertex's core number; an isolated vertex has core number 0.

Queries must use the precomputed index rather than recomputing the core decomposition. Traversing an indexed component to enumerate its vertices is allowed, including when the answer contains every vertex. Analyse both index access and the cost of producing the required sorted list.

Valid inputs and boundary cases

The constructor receives a non-negative integer n and a list of distinct undirected edges, with no self-loops. Vertex IDs are 0..n-1. Query k is any non-negative integer. The **0-core is G itself**, so query(0, v) returns v's ordinary connected component. If n = 0, the edge list is empty and no vertex queries are issued. Calls occur after build(); invalid-input handling is not assessed.

Figure 1 and worked examples



Two K4 subgraphs, a triangle {3,4,5}, bridge 5-6 and pendant vertex 10.

Vertices 0,1,2,3 and 6,7,8,9 have core number 3; vertices 4 and 5 have core number 2; vertex 10 has core number 1.

```
query(1, 0)  -> [0,1,2,3,4,5,6,7,8,9,10]
query(2, 4)  -> [0,1,2,3,4,5,6,7,8,9]
query(3, 3)  -> [0,1,2,3]
query(3, 7)  -> [6,7,8,9]
query(4, 0)  -> []
query(2, 10) -> []
max_k(0, 7)  -> 2
max_k(0, 4)  -> 2
max_k(4, 10) -> 1
max_k(1, 2)  -> 3
max_k(10,10) -> 1
```

The two K4 subgraphs are different connected 3-cores. A core number alone does not identify which connected component contains a vertex.

Implementation

Use Q2.ipynb and implement the three methods in the KCoreIndex cell. The constructor already receives n and the edge list and creates an adjacency representation. You may change the internal representation and add helpers, but keep public signatures and input/output formats unchanged. Put helper definitions and imports before the local-test cell.

Use only the Python Standard Library. The student notebook is a template: its TODOs and placeholder returns are intentional. Do not change the test cell.

Testing and expected scale

Local tests check the figure, boundary cases and random small graphs against brute-force results. The scale test has **10,000 vertices and 80,000 edges**, with 2,000 query calls and 20,000 max_k calls. Deterministic fixtures and SHA-256 digests check the complete ordered outputs, including vertex membership.

Hidden tests may be larger, up to **100,000 vertices and 800,000 edges**. They can include disconnected graphs, isolated vertices, identical endpoints, high core numbers, and both small and large answers. Passing the local tests does not establish the claimed complexity.

Supporting document: Q2.pdf

Explain the index design and justify correctness for both query types. Analyse the **whole construction**, including core decomposition and all subsequent index work; the time for each query type; and index space. Include any ordering or sorting cost in query(k, v).

Use n, m, d(v), k_max and |A| for vertex count, edge count, original-graph degree, maximum core number and answer size. You may also use maximum/minimum/average degree (average 2m/n when n > 0). For restricted traversals, count examined edges, including edges to excluded vertices.

Report **additional retained index space** separately from the original graph representation, peak construction working space and returned output. Retaining adjacency does not itself violate the index-space convention; disclose its O(n+m) storage.

Submission and marking

Component	Marks
Correctness: query and max_k	4
Efficiency: entire construction, both query types and index space	8
Time and space complexity analysis	3

Submit **Q2.ipynb and Q2.pdf** on Moodle. Restart the notebook session and run all cells in order before submission. Correct alternative index designs are accepted; their efficiency is assessed using their actual costs.

END OF QUESTION